PostgreSQL at Scale

Handling Large Data Sets | Assignment 1 | Experimental report | 29 October 2026

Abstract

This study examines deterministic synthetic event datasets of 100,000 and 1,000,000 rows in PostgreSQL 14.18. It evaluates CSV ingestion, validation, five query workloads, two B-tree indexes, storage, and repeated import overhead. Selective indexed queries improved markedly in sampled client-side medians, while two secondary indexes raised median 100K import time by 2.457×. A supplementary pilot comparison exposed a Q5 result-cardinality confound; a separately instrumented PK-only follow-up documents that limitation rather than masking it.

1. Environment and dataset

Component	Recorded configuration
Platform	MacBook Pro, Apple M4 Pro (12 CPU cores), 24 GB RAM; macOS 26.1
Software	Python 3.9.6; PostgreSQL 14.18 (Homebrew ARM64); psql
Database / seed	bigdata_lab; deterministic generator seed 2026
Selected settings	shared_buffers 128 MiB; work_mem 4 MiB; effective_cache_size 4 GiB; max_parallel_workers_per_gather 2

The eight-column events table stores event ID, UTC timestamp, customer and product identifiers, region, quantity, integer price, and status. Primary-key, NOT NULL, and domain CHECK constraints support validation. CSV was imported using psql \copy; JSONL was generated but not loaded.

2. Experimental sequence

Meeting 1: 100K generation, inspection, schema, import and validation. **Meeting 2:** 1M generation, import, five SQL workloads and PK-only baseline. **Meeting 3:** execution plans, sequential introduction of two indexes and indexed benchmarks. **Meeting 4:** five repeated imports per scratch-table index condition, with alternating order. A supplementary 100K query benchmark and Q5 follow-up were performed later.

3. Import and validation

Observed single-run client-side \copy durations: **224.680 ms** for 100K and **1,666.363 ms** for 1M (ratio approximately 7.42× for 10× rows). Both datasets passed recorded row-count, unique-ID, timestamp and status validation. These isolated imports do not establish a general scaling law.

Workloads and query benchmarking

4. Query design

Query	Workload	Primary access characteristic
Q1	Point lookup by event_id	Primary-key lookup
Q2	Completed revenue by UTC month and region	Large scan, sort and grouping
Q3	Top 10 products by completed revenue	Scan and aggregation
Q4	Count events in March 2025 UTC	Timestamp range predicate
Q5	History of customer_id = 50000	Selective filter and descending sort

5. Benchmark protocol

For each Q1–Q5 client benchmark, one warm-up preceded three measured runs in one psql session; the reported statistic is their median, excluding warm-up. Baseline and final indexed workloads retained the same SQL and parameters. psql \timing measures client-observed elapsed time; separate EXPLAIN (ANALYZE, BUFFERS) reports server-side instrumented execution. The two timing types must not be mixed.

6. Indexes and execution plans

Customer history: idx_events_customer_ts_id on (customer_id, event_ts DESC, event_id DESC). **Time range:** idx_events_event_ts on (event_ts). The original primary-key B-tree remained in both configurations.

Before indexing, Q4 used a parallel sequential scan and Q5 a parallel sequential scan plus sort. In captured final plans, Q4 used an index-only scan (zero heap fetches) and Q5 used bitmap index/heap scans followed by a small quicksort. Q2 still scanned broadly and used an external merge sort (~28,288 kB); Q3 still used parallel scanning and aggregation. Plan changes explain why the selective predicates benefited more than broad aggregations.

7. Original 1M baseline versus final indexed timings

Query	PK-only median	Indexed median	Interpretation
Q1	0.061 ms	0.062 ms	Effectively unchanged
Q2	260.336 ms	262.466 ms	No material benefit
Q3	38.874 ms	37.244 ms	Small inconclusive difference
Q4	17.839 ms	2.210 ms	8.07× sampled speedup
Q5	19.353 ms	0.069 ms	280.48× sampled speedup

The speedups are ratios of three-run client-side medians, not statistically tested production-performance guarantees. The observed Q5 index plan still required sorting because bitmap access did not preserve the requested ordering.

Storage, writes and two-scale comparison

8. Index storage overhead

Structure	Before	After
Events table	80 MB	80 MB
All indexes	21 MB	76 MB
Total relation	101 MB	156 MB

The customer-history index occupied approximately 39 MB and the timestamp index 16 MB. Secondary indexes added roughly **55 MB** of rounded PostgreSQL-reported storage. The size values are rounded rather than exact byte totals.

9. Repeated 100K bulk-import experiment

Condition A retained only the primary key; condition B retained the primary key plus both secondary indexes. The same 100K CSV was imported into scratch tables five times per condition, with order AB, BA, AB, BA, AB. Each trial verified COPY 100000 and the row count; the original 1M events table was not modified.

Round	A: PK only (ms)	B: PK + 2 (ms)
1	213.980	830.925
2	202.670	473.743
3	192.820	475.926
4	133.752	315.061
5	124.535	318.984
Median	192.820	473.743

Ratio of medians: **473.743 / 192.820** ≈ **2.457×**, equivalent to a **145.69%** relative increase. B was slower in all five observed rounds. This descriptive result is sensitive to cache, order and background activity; the first B run was especially high. The timing excludes separate truncation, connection setup and verification.

10. Supplementary 100K versus historical 1M PK-only baseline

Query	100K median (ms)	1M median (ms)	1M / 100K
Q1	0.091	0.061	0.670×
Q2	24.805	260.336	10.495×
Q3	8.147	38.874	4.772×
Q4	2.446	17.839	7.293×
Q5	2.093	19.353	9.247×*

***Critical Q5 caveat:** customer_id = 50000 returned **zero** pilot rows versus **15** full-scale rows. The original 9.247× arithmetic ratio is retained for traceability, but it is **not** an isolated dataset-size effect. Q1 also used different event IDs at the two scales. All ratios are descriptive.

Controlled Q5 follow-up and conclusions

11. Matched-customer, PK-only Q5 follow-up

To address the original zero-versus-15-match confound, customer_id = **66808** was selected because it occurred at both sizes: **8 matches at 100K** and **14 at 1M**. The pilot table and a separately prepared, analyzed 1M scratch copy both had only a primary-key index. The original indexed events table was left unchanged.

Both follow-up queries selected event_id, customer_id and event_ts, ordered by event_ts DESC and event_id DESC, and applied LIMIT 50. This is a **Q5-style variant**, not the original seven-column, unlimited Q5. One warm-up and three instrumented EXPLAIN (ANALYZE, BUFFERS) executions were recorded per table; all pilot runs preceded all full-scale runs.

Execution	100K PK-only (ms)	1M PK-only copy (ms)
Warm-up (excluded)	9.589	21.029
Run 1	9.239	16.103
Run 2	7.222	13.653
Run 3	3.583	11.826
Median	7.222	13.653

The ratio of server-side medians was **13.653 / 7.222 ≈ 1.8905×**, with a 6.431 ms absolute median difference. The 100K plan used a sequential scan and sort; the 1M plan used a parallel sequential scan and sorting. Different match counts, parallelism, physical layout, cache state, and grouped execution order prevent causal isolation of table size. The server-side 1.890× ratio is **not directly comparable** to the original client-side 9.247× ratio.

12. Limitations and implications

Results describe one local machine, deterministic synthetic data, three measured query executions per condition and five imports per index condition. No cold-cache protocol, concurrent workload, inferential significance test, or end-to-end drop/load/rebuild index strategy was measured. Indexing decisions should weigh workload-specific read gains against extra storage and write-maintenance cost.

13. Evidence and submission status

Reproduction evidence includes the unmodified generator, schema and benchmark SQL, original plan and timing captures, results/pilot_100k/, Meeting 4 measurements, logs/q5_controlled_comparison.sql, logs/q5_controlled_comparison.txt, and the 33-section logs/progress_log.md. The SQL follow-up requires its pilot and PK-only scratch tables to exist. Instructor demonstrations/checkpoints are unconfirmed; no instructor sign-off is claimed.

Conclusion: Targeted B-tree indexes substantially improved two selective read workloads in this experiment, while increasing rounded storage by ~55 MB and the sampled median 100K bulk-import duration by ~2.457×. The additional Q5 analysis documents why cross-scale ratios require careful control of cardinality and measurement scope.